FVN and TLN Corrections, Line by Line

An implementation-level manual for GradientFlowRGToolkit

GradientFlowRGToolkit

31 August 2026

Contents

1 What this document explains

This manual documents the two normalization corrections implemented in `src/gfrgtoolkit/stages/tree_level`:

- `fvn.py`: the continuum finite-volume normalization (FVN);
- `tln.py`: the finite-volume, finite-lattice-spacing tree-level normalization (TLN).

It is deliberately more detailed than the shorter *Finite-Volume and Tree-Level Normalization Corrections* guide. The purpose is to make every scientifically meaningful calculation visible at code level. In particular, the manual answers:

1. What does the returned `delta` mean, and why does processing divide by `1 + delta`?
2. Which parts of FVN are printed in the 2012 paper, and which parts are the rectangular-torus and modular-series toolkit implementation?
3. How does TLN translate the 2014 paper's momentum-space kernels and trace into NumPy arrays?
4. Why are momentum-orbit reduction, spectral decomposition, exponent collapsing, direct requested-time evaluation, and caching mathematically legitimate?
5. Which of those steps are published physics and which are numerical toolkit choices?
6. Under what limit should TLN converge to FVN?
7. Which applicability restrictions are enforced, and which are merely documented today?

Code excerpts reflect the implementation on 31 August 2026, with long lines wrapped for print. The source files remain authoritative if the implementation and this document ever diverge.

2 Shared correction convention

2.1 The quantity returned by both modules

At leading order, the continuum infinite-volume gradient-flow energy density has the normalization

$$\langle t^2 E(t) \rangle = g^2 \frac{3(N_c^2 - 1)}{128\pi^2} + O(g^4).$$

Finite volume or finite lattice spacing modifies the tree-level coefficient by a dimensionless factor $C(t)$:

$$\langle t^2 E(t) \rangle = g^2 \frac{3(N_c^2 - 1)}{128\pi^2} C(t) + O(g^4).$$

Both correction modules use the same delta convention:

$$\Delta(t) = C(t) - 1, \qquad C(t) = 1 + \Delta(t). \tag{1}$$

`CorrectionEstimate.delta` stores Δ, not C and not $1/C$:

```python
@dataclass(frozen=True)
class CorrectionEstimate:
    """A correction factor minus one and its evidence."""

    delta: np.ndarray
    evidence: CorrectionEvidence
```

2.2 Where the division occurs

The processing stage asks the configured `CorrectionMethod` to estimate one `CorrectionRequest` per channel, then divides the standard coupling normalization by $1 + \Delta = C$:

```python
correction = configuration.correction.method.estimate(
    CorrectionRequest(
        flow_times=tuple(float(time) for time in selected_times),
        volume=entry.key.volume,
        flow_action=entry.flow,
        gauge_action=gauge_action,
        energy_density_operator=observable,
    )
)

normalization = (
    128.0 * np.pi * np.pi
    / (3.0 * (nc * nc - 1.0))
    * selected_times
    * selected_times
    / (1.0 + correction.delta)
)
coupling = normalization * energy
```

Consequently,

$$g^2_{\mathrm{GF}}(t) = \frac{128\pi^2}{3(N_c^2-1)} \frac{t^2\langle E(t)\rangle}{C(t)}. \tag{2}$$

This protocol is the correction seam. `setup.py` maps legacy enum choices to `FiniteVolumeNormalization` or `FiniteLatticeTreeLevelNormalization` adapters, but neither scientific module imports configuration definitions from `setup.py`. New correction methods can be injected without adding a branch to processing.

The correction is deterministic. Multiplying the already-correlated energy `gvar` values by `normalization` propagates their full covariance exactly; the correction does not estimate or alter Monte Carlo autocorrelation.

2.3 Shared immutable evidence

Each method also returns applicability and provenance:

```
@dataclass(frozen=True)
class CorrectionEvidence:
    method: str
    source: str
    volume: str
    flow_action: str | None
    gauge_action: str | None
    energy_density_operator: str | None
    flow_time_units: str
    interpolation_spacing: float | None
    validity_domain: str | None = None
    numerical_tolerance: float | None = None
    implementation_notes: tuple[str, ...] = ()
```

`None` means that a discretization field does not apply to that method. FVN is a continuum factor, so its flow action, simulation gauge action, and energy-density operator are `None`. TLN depends on all three and records them. Both methods record the domain they enforce and the numerical tolerance that controls their approximation. `report_description` renders these fields in the processing report rather than reducing evidence to a bare method name.

3 FVN: continuum finite-volume normalization

3.1 Published hypercubic formula

For a periodic four-torus with equal extent L in all directions, Fodor et al. write

$$\langle t^2 E(t)\rangle = g_R^2 \frac{3(N_c^2-1)}{128\pi^2}\left[1+\delta(t,L)\right].$$

Their Eqs. (1.2)–(1.3) separate the correction into an algebraic zero-mode piece and an exponential nonzero-mode piece:

$$\delta(t,L) = \delta_a(t,L) + \delta_e(t,L),$$

$$\delta_a(t,L) = -\frac{64\pi^2 t^2}{3L^4}, \qquad \delta_e(t,L) = \vartheta_3\left(e^{-L^2/(8t)}\right)^4 - 1. \tag{3}$$

The Jacobi function is

$$\vartheta_3(q) = \sum_{n=-\infty}^{\infty} q^{n^2} = 1 + 2\sum_{n=1}^{\infty} q^{n^2}. \tag{4}$$

The authority is Fodor et al., *The Yang–Mills gradient flow in finite volume*, DOI 10.1007/JHEP11(2012)007, arXiv:1208.1051.

In terms of the finite-volume scheme parameter

$$c = \frac{\sqrt{8t}}{L},$$

the same hypercubic factor is

$$C_{\mathrm{FVN}}(c) = \vartheta_3(e^{-1/c^2})^4 - \frac{\pi^2}{3}c^4. \tag{5}$$

3.2 Geometry string parsing

The shared `LatticeGeometry` value parses the complete canonical volume string. Both FVN and TLN therefore accept exactly the same syntax:

```
match = re.fullmatch(r"l(\d+)l(\d+)l(\d+)t(\d+)", volume)
if match is None:
    raise TreeLevelCorrectionError(
        "volume must have form l24l24l24t48"
    )
extents = tuple(int(value) for value in match.groups())
if any(value <= 0 for value in extents):
        raise TreeLevelCorrectionError(
            "volume must contain four positive extents"
        )
```

For `l24l24l24t48`, the result is

$$(L_0, L_1, L_2, L_3) = (24, 24, 24, 48).$$

The full match rejects missing fields, extra prefixes or suffixes, and zero extents at the correction boundary. Geometry semantics therefore do not depend on how a particular dataset catalog happens to name files.

3.3 Input time validation

FVN accepts a one-dimensional vector of positive finite flow times:

```
times = np.asarray(flow_times, dtype=float)
if (
    times.ndim != 1
    or times.size == 0
    or not np.all(np.isfinite(times))
    or np.any(times <= 0.0)
):
    raise TreeLevelCorrectionError(
        "finite-volume normalization requires "
        "positive finite flow times"
    )
```

The function does not require sorted or distinct times because each value is evaluated independently. It accepts every positive finite time because the complete theta function is evaluated to a declared tolerance.

3.4 Rectangular-torus algebraic term, line by line

The implementation generalizes L^4 to the four-volume $\prod_\mu L_\mu$:

$$\Delta_a(t; L_0, \dots, L_3) = -\frac{64\pi^2 t^2}{3\prod_{\mu=0}^{3} L_\mu}. \tag{6}$$

Rather than writing Eq. (6) directly, the code builds it one direction at a time:

```
algebraic = np.full_like(
    times,
    -64.0 * np.pi**2 / 3.0,
)
for extent in extents:
    ratio = extent * extent / times
    algebraic /= np.sqrt(ratio)
```

For one direction,

$$\frac{1}{\sqrt{L_\mu^2/t}} = \frac{\sqrt{t}}{L_\mu}.$$

After four loop iterations,

$$-\frac{64\pi^2}{3}\prod_{\mu=0}^{3}\frac{\sqrt{t}}{L_\mu} = -\frac{64\pi^2 t^2}{3\prod_\mu L_\mu},$$

which is Eq. (6). On a hypercube, $\prod L_\mu = L^4$, so it reduces exactly to the paper's algebraic term.

The rectangular product is a natural separable-torus extension retained from `ContinuousBetaFunction`; it is not the geometry printed in the paper's displayed hypercubic formula. The evidence names this distinction.

3.5 Complete Jacobi-theta product, line by line

For each direction, define

$$q_\mu = e^{-L_\mu^2/(8t)}.$$

The one-dimensional factor required by Eq. (4) is

$$\vartheta_3(q_\mu) = 1 + 2\sum_{n=1}^{\infty} e^{-n^2 L_\mu^2/(8t)}. \tag{7}$$

Writing $x = L_\mu^2/(8t)$, direct summation converges rapidly for $x \geq \pi$. For smaller x, `_theta3_exp_minus` applies

$$\vartheta_3(e^{-x}) = \sqrt{\frac{\pi}{x}}\,\vartheta_3(e^{-\pi^2/x}). \tag{8}$$

The transformed exponent is at least π, so one of the two rapidly convergent forms always applies:

```
use_modular_form = exponent < np.pi
reduced_exponent = np.where(
    use_modular_form,
    np.pi**2 / exponent,
```

```
    exponent,
)
prefactor = np.where(
    use_modular_form,
    np.sqrt(np.pi / exponent),
    1.0,
)
theta = np.ones_like(exponent)
for index in range(1, 65):
    term = 2.0 * np.exp(-reduced_exponent * index * index)
    theta += term
    if np.all(term <= relative_tolerance * theta):
        return prefactor * theta
```

The stopping test is simultaneous across the requested vector. If 64 terms do not reach the declared relative tolerance, the method raises `TreeLevelCorrectionError` rather than silently returning a partial sum. The public default is 10^{-15}, validated as positive and finite.

The four-dimensional product is assembled explicitly:

```
exponential = np.ones_like(times)
for extent in extents:
    ratio = extent * extent / times
    exponential *= _theta3_exp_minus(
        ratio / 8.0,
        relative_tolerance=theta_relative_tolerance,
    )
```

An independent test compares the implementation with a direct high-precision series on a hypercube. The $c = 2$ value is especially discriminating because the inherited $|n| \leq 2$ approximation was no longer accurate:

$$C_{\mathrm{FVN}}(c = 2) = 105.2757802782865.$$

This is a code-validation oracle derived from Eq. (5), not a literature datum and not a tolerance inferred from an external analysis dataset.

3.6 Final FVN value and evidence

The code combines the algebraic term and the complete theta product:

```
return CorrectionEstimate(
    delta=algebraic + exponential - 1.0,
    evidence=CorrectionEvidence(
        method="finite-volume-normalization",
        source=(
            "https://doi.org/10.1007/"
            "JHEP11(2012)007"
        ),
        volume=volume,
        flow_action=None,
        gauge_action=None,
        energy_density_operator=None,
        flow_time_units="t/a^2",
        interpolation_spacing=None,
        validity_domain="positive finite flow time",
        numerical_tolerance=float(theta_relative_tolerance),
        implementation_notes=(
            "rectangular-torus product extension",
```

```
            "Jacobi theta modular-series evaluation",
        ),
    ),
)
```

Since `exponential` evaluates the full theta product, subtracting one turns it into Δ_e. Thus

$$\Delta_{\mathrm{FVN}} = \Delta_a + \Delta_e = \texttt{algebraic} + \texttt{exponential} - 1.$$

The source DOI supports the continuum hypercubic formula. Evidence separately identifies the rectangular extension, modular-series implementation, validity domain, and numerical tolerance.

3.7 FVN numerical example

For an 8^4 box and $t/a^2 = (0.5, 1, 2)$:

```
estimate = finite_volume_correction(
    np.array([0.5, 1.0, 2.0]),
    volume="181818t8",
)
print(estimate.delta)
```

the current implementation gives

```
[-0.01285015 -0.04871779 -0.05084113]
```

and therefore

```
C = 1 + delta
  = [0.98714985 0.95128221 0.94915887]
```

At $t = 1$, for example, the algebraic term is

$$-\frac{64\pi^2}{3\,8^4} \approx -0.0514042,$$

while the small positive theta contribution brings the total delta to about -0.0487178.

4 TLN: finite-volume, finite-spacing normalization

4.1 Published master formula

The 2014 calculation keeps the flow, simulation action, and energy observable as separate lattice kernels:

- $\mathcal{S}^f$: gradient-flow action kernel;
- $\mathcal{S}^g$: dynamical gauge-action kernel;
- $\mathcal{S}^e$: energy-density measurement kernel;
- $\mathcal{G}$: gauge-fixing kernel.

Published Eq. (6.2) is displayed for a periodic hypercubic lattice with four-volume L^4. The implementation uses the separable $N_s^3 N_t$ extension and, in lattice units $a = 1$, evaluates

$$C(t, L) = \frac{128\pi^2 t^2}{3V} + \frac{64\pi^2 t^2}{3V} \sum_{p \neq 0} \mathrm{Tr}\left[e^{-t(\mathcal{S}^f + \mathcal{G})} (\mathcal{S}^g + \mathcal{G})^{-1} e^{-t(\mathcal{S}^f + \mathcal{G})} \mathcal{S}^e \right], \tag{8}$$

with periodic momenta

$$p_\mu = \frac{2\pi n_\mu}{N_\mu}, \qquad V = \prod_\mu N_\mu.$$

The first term in Eq. (8) is the zero-mode contribution. The momentum sum excludes $p = 0$. Replacing the paper's L^4 by $V = N_s^3 N_t$ is a derived rectangular-time extension, not a distinct formula printed in the paper.

The authority is Fodor et al., *The lattice gradient flow at tree-level and its improvement*, DOI 10.1007/JHEP09(2014)018, arXiv:1406.0827v2.

4.2 Implemented applicability matrix

The coefficient tables encode the supported discretizations:

```
_ENERGY_KERNEL = {
    "p": 0.0,
    "s": -1.0 / 12.0,
    "c": 999.0,
}
_GAUGE_KERNEL = {
    "s": -1.0 / 12.0,
    "w": 0.0,
}
```

Their meanings are:

Interface value	Kernel	Physical meaning
gauge action `w`	$c_g = 0$	Wilson simulation action
gauge action `s`	$c_g = -1/12$	tree-level Symanzik simulation action
observable `p`	$c_e = 0$	plaquette energy density
observable `s`	$c_e = -1/12$	tree-level Symanzik energy density
observable `c`	private sentinel `999.0`	clover energy kernel, not an action coefficient

The flow kernel is fixed to Wilson flow, $c_f = 0$. This appears twice:

```
coefficients, exponents = _spectral_terms(
    ...,
    flow_kernel=0.0,
    gauge_kernel=gauge_kernel,
    energy_kernel=energy_kernel,
)
```

and at the public evidence boundary:

```
if flow_action != "wilson":
    raise TreeLevelCorrectionError(
        "tree-level normalization supports only Wilson flow"
    )
```

Rejecting unsupported flow actions is essential. Reusing the Wilson flow kernel while merely relabeling it as Zeuthen or Symanzik flow would compute a scientifically different correction.

4.3 TLN input validation and geometry

The public numerical function validates the action/operator codes, a nonempty positive finite one-dimensional time array, and a positive finite exponent-collapse tolerance:

```
if observable not in _ENERGY_KERNEL:
    raise TreeLevelCorrectionError(
        f"unsupported energy-density operator {observable!r}"
```

```
    )
if gauge_action not in _GAUGE_KERNEL:
    raise TreeLevelCorrectionError(
        f"unsupported gauge action {gauge_action!r}"
    )

times = np.asarray(flow_times, dtype=float)
if (
    times.ndim != 1
    or times.size == 0
    or not np.all(np.isfinite(times))
    or np.any(times <= 0.0)
):
    raise TreeLevelCorrectionError(
        "tree-level normalization requires "
        "positive finite flow times"
    )
if (
    not np.isfinite(exponent_collapse_tolerance)
    or exponent_collapse_tolerance <= 0.0
):
    raise TreeLevelCorrectionError(
        "exponent_collapse_tolerance must be "
        "positive and finite"
    )
```

TLN allows a temporal extent different from the three spatial extents but requires isotropic space:

```
values = LatticeGeometry.parse(volume).extents
if len(set(values[:3])) != 1:
    raise TreeLevelCorrectionError(
        "tree-level normalization requires equal spatial extents"
    )
if int(np.prod(values)) <= 1:
    raise TreeLevelCorrectionError(
        "tree-level normalization requires "
        "at least one nonzero momentum"
    )
return values[0], values[-1]
```

Thus the supported volume is $N_s^3 N_t$. Equal spatial axes are not a physics requirement of Eq. (8); they are an implementation restriction that enables the spatial permutation-orbit reduction below.

5 TLN momentum-orbit reduction

5.1 Why a reduced sum is allowed

The direct nonzero sum contains $N_s^3 N_t - 1$ momenta. For equal spatial extents, the trace is invariant under:

- reflection of any momentum component;
- permutation of the three spatial components.

`_momentum_orbits` keeps one representative and attaches its exact multiplicity. This reorganizes the finite sum without approximating it.

5.2 Reflection representatives

Each component is reduced to $0, \ldots, \lfloor N_\mu/2 \rfloor$:

```
spatial = np.arange(spatial_extent // 2 + 1)
temporal = np.arange(temporal_extent // 2 + 1)
spatial_reflections = np.where(
    (spatial == 0)
    | (2 * spatial == spatial_extent),
    1,
    2,
)
temporal_reflections = np.where(
    (temporal == 0)
    | (2 * temporal == temporal_extent),
    1,
    2,
)
```

Zero is its own reflection and receives weight one. On an even extent, the Nyquist component $N_\mu/2$ is also its own reflection and receives weight one. Every other representative stands for the pair n and $N_\mu - n$ and receives weight two. For odd extents there is no Nyquist fixed point, and the largest retained component correctly receives weight two.

5.3 Sorted spatial triples and permutation weight

The code forms all reduced spatial triples, then keeps $n_x \le n_y \le n_z$:

```
first, second, third = np.meshgrid(
    spatial,
    spatial,
    spatial,
    indexing="ij",
)
keep = (first <= second) & (second <= third)
first = first[keep]
second = second[keep]
third = third[keep]
```

The number of distinct permutations is one, three, or six:

```
first_equals_second = first == second
second_equals_third = second == third
permutations = np.where(
    first_equals_second & second_equals_third,
    1,
    np.where(
        first_equals_second | second_equals_third,
        3,
        6,
    ),
)
```

- (a, a, a) has one permutation;
- (a, a, b) or (a, b, b) has three;
- (a, b, c) with all distinct has six.

The spatial multiplicity is the permutation count times the three reflection counts. It is then multiplied by the temporal reflection count.

5.4 Zero-mode removal and closure invariant

Representatives and weights are flattened. The first representative is $(0,0,0,0)$ and is removed because its analytic contribution is the first term in Eq. (8):

```
representatives = representatives[1:]
multiplicities = multiplicities[1:]
expected = spatial_extent**3 * temporal_extent - 1
if int(multiplicities.sum()) != expected:
    raise TreeLevelCorrectionError(
        "tree-level momentum-orbit multiplicities do not close"
    )
```

The equality

$$\sum_{r\in\text{orbits}} w_r = N_s^3 N_t - 1 \tag{9}$$

is an exact runtime proof that no nonzero momentum was lost or double-counted by the combinatorial reduction.

For an 8^4 box, the full grid has 4095 nonzero momenta. The code reduces them to 174 representatives whose multiplicities sum to 4095. Some first representatives and weights are:

```
(0,0,0,1) -> 2      (0,0,0,4) -> 1
(0,0,1,0) -> 6      (0,0,1,1) -> 12
```

For example, $(0,0,1,0)$ has three spatial placements times two reflections, giving weight six.

6 TLN kernel construction

6.1 Lattice momenta

For each orbit block, the code constructs

```
extents = np.array(
    [
        spatial_extent,
        spatial_extent,
        spatial_extent,
        temporal_extent,
    ],
    dtype=float,
)
lattice_momentum = 2.0 * np.pi * indices / extents
half_momentum = 2.0 * np.sin(lattice_momentum / 2.0)
half_squared = half_momentum**2
half_norm = half_squared.sum(axis=1)
momentum = np.sin(lattice_momentum)
momentum_norm = (momentum**2).sum(axis=1)
cosine_half = np.cos(lattice_momentum / 2.0)
```

These arrays are the paper's lattice momenta at $a = 1$:

$$\hat{p}_\mu = 2\sin(p_\mu/2), \qquad \tilde{p}_\mu = \sin(p_\mu),$$

$$\hat{p}^2 = \sum_\mu \hat{p}_\mu^2, \qquad \tilde{p}^2 = \sum_\mu \tilde{p}_\mu^2.$$

The leading array axis indexes momentum representatives; the final two axes of each kernel are the Lorentz indices $\mu, \nu = 0, 1, 2, 3$.

6.2 Symanzik-family action kernel

Published Eq. (3.5), at $a = 1$, is

$$S_{\mu\nu}(c) = \delta_{\mu\nu}\left[\hat{p}^2 - c\sum_\rho \hat{p}_\rho^4 - c\hat{p}_\mu^2\hat{p}^2\right] - \hat{p}_\mu\hat{p}_\nu\left[1 - c(\hat{p}_\mu^2 + \hat{p}_\nu^2)\right]. \tag{10}$$

The off-diagonal-and-common part is initialized first:

```
matrix = (
    -half_momentum[:, :, None]
    * half_momentum[:, None, :]
    * (
        1.0
        - coefficient
        * (
            half_squared[:, :, None]
            + half_squared[:, None, :]
        )
    )
)
```

For one momentum, `half_momentum[:, :, None]` has components $\hat{p}_\mu$ down the row axis and `half_momentum[:, None, :]` has $\hat{p}_\nu$ across the column axis. Their product implements the second term of Eq. (10) for every μ, ν.

The Kronecker-delta term is then added to the four diagonal entries:

```
matrix[:, _DIRECTIONS, _DIRECTIONS] += (
    half_norm[:, None]
    - coefficient
    * np.sum(half_squared**2, axis=1)[:, None]
    - coefficient
    * half_squared
    * half_norm[:, None]
)
```

Here `np.sum(half_squared**2, axis=1)` is $\sum_\rho \hat{p}_\rho^4$, and the final broadcasted product is $\hat{p}_\mu^2\hat{p}^2$.

Setting $c = 0$ yields the Wilson kernel. Setting $c = -1/12$ yields the tree-level Symanzik kernel.

6.3 Clover energy kernel

The clover observable uses published Eq. (3.6):

$$K_{\mu\nu} = \left(\delta_{\mu\nu}\tilde{p}^2 - \tilde{p}_\mu\tilde{p}_\nu\right)\cos(p_\mu/2)\cos(p_\nu/2). \tag{11}$$

The code builds the transverse bracket and multiplies the cosine factors:

```
matrix = (
    -momentum[:, :, None]
    * momentum[:, None, :]
)
matrix[:, _DIRECTIONS, _DIRECTIONS] += (
    momentum_norm[:, None]
)
```

```
return (
    matrix
    * cosine_half[:, :, None]
    * cosine_half[:, None, :]
)
```

The value `999.0` is checked only inside `_kernel` to dispatch here:

```
if coefficient == 999.0:
    return _clover_kernel(
        momentum,
        momentum_norm,
        cosine_half,
    )
return _action_kernel(...)
```

It has no physical meaning and never enters Eq. (10).

6.4 Gauge fixing and the three matrices

With gauge parameter $\alpha = 1$, Eq. (3.10) gives

$$G_{\mu\nu} = \hat{p}_\mu \hat{p}_\nu.$$

The implementation forms it as an outer product:

```
gauge_fixing = (
    half_momentum[:, :, None]
    * half_momentum[:, None, :]
)
flow_matrix = _kernel(flow_kernel, ...) + gauge_fixing
gauge_matrix = _kernel(gauge_kernel, ...) + gauge_fixing
energy_matrix = _kernel(energy_kernel, ...)
```

Gauge fixing is added to the flow and inverse-action kernels, not to the transverse energy observable. The paper shows the final result is independent of the gauge-fixing parameter; the implementation fixes $\alpha = 1$ rather than treating it as a public analysis choice.

7 TLN spectral trace reduction

7.1 Matrix identity

For one nonzero momentum, define

$$F = \mathcal{S}^f + \mathcal{G}, \qquad P = \mathcal{S}^g + \mathcal{G}, \qquad O = \mathcal{S}^e.$$

F is real symmetric, so

$$F = V \Lambda V^\mathsf{T}, \qquad \Lambda = \mathrm{diag}(\lambda_0, \dots, \lambda_3).$$

Define the transformed matrices

$$A = V^\mathsf{T} P^{-1} V, \qquad B = V^\mathsf{T} O V.$$

Cyclic invariance of the trace and the diagonal form of $e^{-t\Lambda}$ give

$$\mathrm{Tr}\left[e^{-tF}P^{-1}e^{-tF}O\right] = \sum_{i,j=0}^{3} A_{ij}B_{ji}e^{-t(\lambda_i+\lambda_j)}. \tag{12}$$

Equation (12) converts one matrix trace at every requested time into 16 scalar exponentials whose coefficients can be prepared once.

7.2 Code mapping

The flow kernel is diagonalized in a vectorized batch:

```
eigenvalues, eigenvectors = np.linalg.eigh(flow_matrix)
transpose = np.swapaxes(eigenvectors, 1, 2)
```

NumPy stores eigenvectors in columns, so `transpose` is V^{T}. The transformed inverse action is computed with a linear solve:

```
inverse_action = (
    transpose
    @ np.linalg.solve(gauge_matrix, eigenvectors)
)
```

`np.linalg.solve(P, V)` calculates $P^{-1}V$ without explicitly forming P^{-1}; left multiplication by V^{T} gives A.

The energy kernel is transformed similarly:

```
measured_energy = (
    transpose @ (energy_matrix @ eigenvectors)
)
```

This is $B = V^{\mathsf{T}}OV$.

Finally, the code stores the coefficient and exponent of every i, j pair:

```
coefficients.append(
    (
        inverse_action
        * np.swapaxes(measured_energy, 1, 2)
        * weights[:, None, None]
    ).ravel()
)
exponents.append(
    (
        eigenvalues[:, :, None]
        + eigenvalues[:, None, :]
    ).ravel()
)
```

The axis swap on `measured_energy` converts B_{ij} to B_{ji}, exactly as required by Eq. (12). Orbit multiplicity w is folded into each coefficient:

$$a_{pij} = w_p A_{ij} B_{ji}, \qquad \mu_{pij} = \lambda_i + \lambda_j.$$

The full nonzero-mode sum is therefore

$$\sum_k a_k e^{-t\mu_k}. \tag{13}$$

Processing representatives in blocks of 200,000 bounds temporary memory. It does not change the mathematical sum.

8 Exponent collapsing and direct requested-time evaluation

8.1 Collapsing equal spectral exponents

Many Wilson-flow momenta produce repeated exponent values. Since

$$a_1 e^{-t\mu} + a_2 e^{-t\mu} = (a_1 + a_2) e^{-t\mu},$$

equal exponents can be merged before evaluating the time grid.

```
order = np.argsort(exponents, kind="stable")
sorted_exponents = exponents[order]
sorted_coefficients = coefficients[order]
starts_new = np.empty(sorted_exponents.size, dtype=bool)
starts_new[0] = True
np.greater(
    np.diff(sorted_exponents),
    tolerance,
    out=starts_new[1:],
)
starts = np.flatnonzero(starts_new)
return (
    sorted_exponents[starts],
    np.add.reduceat(sorted_coefficients, starts),
)
```

The default absolute tolerance is 10^{-11}. Adjacent sorted exponents whose difference is no larger than the tolerance belong to one group. Because the rule compares adjacent values, a chain of individually close exponents can span more than 10^{-11} from the first to last member. This is a numerical toolkit policy, not a step prescribed in the paper.

The stable sort makes the summation order reproducible for a fixed input and software environment. The tolerance should be covered by numerical accuracy tests because merging merely *nearly* equal exponents is an approximation.

8.2 Fixed-order evaluation at requested times

The Numba kernel parallelizes over requested flow times, not over pieces of one reduction:

```
@njit(parallel=True, cache=False)
def _evaluate_spectrum(coefficients, exponents, flow_times):
    values = np.empty(flow_times.shape[0])
    for time_index in prange(flow_times.shape[0]):
        total = 0.0
        for term_index in range(coefficients.shape[0]):
            total += coefficients[term_index] * np.exp(
                -flow_times[time_index] * exponents[term_index]
            )
        values[time_index] = total
    return values
```

Every time therefore evaluates Eq. (13) directly. The inner reduction order is fixed, and Numba fast math is not enabled. Changing the Numba thread count cannot change how the terms for one answer are grouped; the test suite checks exact equality across thread counts.

9 Cached spectrum, zero mode, and exact domain

9.1 Exact public domain

Before calculating a spectrum, `tree_level_delta` checks

$$0 < t \leq \frac{N_s^2}{32}, \qquad \frac{\sqrt{8t}}{N_s} \leq \frac{1}{2}. \tag{14}$$

The bound is computed directly from the spatial extent and compared with the requested maximum. There is no rounded interpolation endpoint, so values slightly outside Eq. (14) cannot leak through. Empty, nonfinite, or nonpositive time arrays are rejected as typed correction errors as well.

9.2 Restoring the analytic zero mode

After evaluating Eq. (13), the code defines

```
prefactor = (
    64.0
    * np.pi**2
    * flow_times**2
    / (
        3.0
        * spatial_extent**3
        * temporal_extent
    )
)
return 2.0 * prefactor + prefactor * lattice_sum
```

Since $V = N_s^3 N_t$, the nonzero part is

$$\frac{64\pi^2 t^2}{3V} \sum_{p \neq 0} T(p, t),$$

and `2.0 * prefactor` is

$$\frac{128\pi^2 t^2}{3V},$$

the analytic zero-mode term in Eq. (8). Removing the zero representative and adding this term are complementary operations; including both the zero representative and `2 * prefactor` would double count it.

9.3 Cache identity

`_tree_level_spectrum` is decorated with

```
@lru_cache(maxsize=64)
```

and its arguments are

```
(spatial_extent, temporal_extent,
 gauge_kernel, energy_kernel,
 exponent_collapse_tolerance)
```

The flow coefficient is absent from the key because the implementation fixes Wilson flow internally. Equal requests reuse immutable collapsed coefficient and exponent arrays; requested values are never cached or interpolated. The cache is a process-local performance mechanism, while `CorrectionEvidence` records the scientific inputs and collapse tolerance rather than cache hits.

9.4 Return value and the small-time oracle

The directly evaluated normalization is checked to be positive and finite. The returned subtraction implements Eq. (1):

$$\texttt{tree\_level\_delta} = C_{\mathrm{TLN}} - 1.$$

The test suite independently enumerates all $4^3 \times 6$ momenta and uses `scipy.linalg.expm` to evaluate the published matrix trace at $t/a^2 = 10^{-4}$. This deliberately targets the region where the former 0.01-grid spline failed. The public result agrees with that independent full sum, and a second parameterized oracle covers both gauge actions and all three energy operators at $t/a^2 = 0.17$.

10 TLN wrapper and evidence

`tree_level_delta` is the numerical function. `tree_level_correction` adds the flow-action applicability check and constructs the scientific record:

```
return CorrectionEstimate(
    delta=tree_level_delta(
        flow_times,
        observable=observable,
        volume=volume,
        gauge_action=gauge_action,
        exponent_collapse_tolerance=(
            exponent_collapse_tolerance
        ),
    ),
    evidence=CorrectionEvidence(
        method=(
            "finite-lattice-tree-level-normalization"
        ),
        source=(
            "https://doi.org/10.1007/"
            "JHEP09(2014)018"
        ),
        volume=volume,
        flow_action=flow_action,
        gauge_action=gauge_action,
        energy_density_operator=observable,
        flow_time_units="t/a^2",
        interpolation_spacing=None,
        validity_domain=(
            "0 < sqrt(8t)/N_s <= 1/2"
        ),
        numerical_tolerance=float(
            exponent_collapse_tolerance
        ),
        implementation_notes=(
            "rectangular-time periodic momentum sum",
            "direct requested-time spectral evaluation",
            "fixed-order per-time reduction without fast math",
        ),
    ),
)
```

The method identity deliberately says **finite-lattice** tree-level normalization. It includes both finite volume and finite spacing; it is not the same quantity as FVN at coarse t/a^2.

11 TLN numerical examples

11.1 Discretization dependence on one lattice

For an 8^4 box and $t/a^2 = (0.5, 1, 2)$, Wilson flow with Wilson gauge action and plaquette energy (`WWP`) gives:

```
tree_level_delta(
    np.array([0.5, 1.0, 2.0]),
    observable="p",
    volume="18l8l8t8",
    gauge_action="w",
)
```

```
[0.41778605 0.11645453 0.04274070]
```

For Wilson flow, tree-level Symanzik gauge action, and clover energy (`WSC`), the same times give:

```
[-0.19106016 -0.14937655 -0.08458950]
```

The difference is expected. TLN depends on the Flow–Action–Observable triple, and applying one triple's correction to another discretization would not be a benign approximation.

11.2 Why the correction can be above or below one

For WWP at $t/a^2 = 0.5$, $\Delta \approx 0.418$, so Eq. (2) divides the raw coupling normalization by about 1.418. For WSC at the same time, $\Delta \approx -0.191$, so it divides by about 0.809. A negative delta is not an invalid correction as long as $C = 1 + \Delta$ remains physically and numerically meaningful.

The implementation explicitly rejects a nonpositive or nonfinite C after direct evaluation. A negative delta is accepted exactly when $1 + \Delta$ remains positive and finite.

12 The precise FVN–TLN convergence limit

12.1 Two dimensionless variables

The 2014 paper writes the finite-lattice factor as a function of

$$x = \frac{a^2}{t}, \qquad c = \frac{\sqrt{8t}}{L}.$$

In lattice units,

$$\frac{t}{a^2} = \frac{c^2}{8}\left(\frac{L}{a}\right)^2. \tag{14}$$

FVN is the continuum finite-volume limit:

$$C_{\mathrm{FVN}}(c) = C_{\mathrm{TLN}}(0, c).$$

Therefore the relevant test is

$$\boxed{\lim_{a^2/t \to 0 \text{ at fixed } c} C_{\mathrm{TLN}}(a^2/t, c) = C_{\mathrm{FVN}}(c)}. \tag{15}$$

Merely increasing t/a^2 on a fixed lattice also changes c, so it does not isolate the finite-spacing limit. To hold c fixed, increase L/a and choose t/a^2 according to Eq. (14).

12.2 Implemented convergence oracle

The public-path test fixes $c = 0.30$ and uses hypercubic extents $L/a = (8, 16, 24, 32)$:

```
scheme_ratio = 0.30
extents = (8, 16, 24, 32)
for extent in extents:
    target_time = (
        (scheme_ratio * extent) ** 2 / 8.0
    )
```

It processes the same synthetic Wilson-flow, Wilson-action, plaquette-energy histories through both public correction policies. At the target times, the normalization factors are:

L/a	t/a^2	C_{FVN}	C_{TLN}	$\lvert C_{\mathrm{FVN}}/C_{\mathrm{TLN}} - 1\rvert$
8	0.72	0.9734716362	1.2281887796	0.2073925016
16	2.88	0.9734716362	1.0202024211	0.0458054048
24	6.48	0.9734716362	0.9934231357	0.0200835865
32	11.52	0.9734716362	0.9845456861	0.0112478782

C_{FVN} is constant because c is fixed. The discrepancy falls monotonically and is below 1.2% at $L/a = 32$. For the same raw energy,

$$\frac{g^2_{\mathrm{TLN}}}{g^2_{\mathrm{FVN}}} = \frac{C_{\mathrm{FVN}}}{C_{\mathrm{TLN}}},$$

so the public test's corrected-coupling comparison is equivalent to the final column.

This oracle simultaneously checks the two independent implementations, processing's correction direction, DOI evidence, and the predicted continuum relationship. Agreement at one volume would be much weaker evidence than the observed convergence sequence.

13 Result reporting

`ProcessingResult.correction_methods` uses the evidence's `report_description`:

`channel.correction.report_description`

Representative report entries are therefore:

```
corrections: finite-volume-normalization
  [source: https://doi.org/10.1007/JHEP11(2012)007;
   domain: positive finite flow time;
   numerical tolerance: 1e-15;
   implementation: rectangular-torus product extension,
   Jacobi theta modular-series evaluation]
```

or

```
corrections: finite-lattice-tree-level-normalization
  [source: https://doi.org/10.1007/JHEP09(2014)018;
   flow: wilson; gauge action: w; energy: p;
   domain: 0 < sqrt(8t)/N_s <= 1/2;
   numerical tolerance: 1e-11;
   implementation: rectangular-time periodic momentum sum,
```

```
  direct requested-time spectral evaluation,
  fixed-order per-time reduction without fast math]
```

For full applicability, inspect channel evidence programmatically:

```
evidence = result.ensembles[0].channels[0].correction
print(evidence.method)
print(evidence.source)
print(evidence.volume)
print(evidence.flow_action)
print(evidence.gauge_action)
print(evidence.energy_density_operator)
print(evidence.flow_time_units)
print(evidence.interpolation_spacing)
print(evidence.validity_domain)
print(evidence.numerical_tolerance)
print(evidence.implementation_notes)
```

The report deduplicates complete descriptions. Applicable Flow–Action–Observable fields, validity domain, tolerance, and implementation notes are therefore visible without inspecting Python objects.

14 Published physics versus toolkit policy

Element	Status
FVN hypercubic zero/nonzero-mode formula	Published in the 2012 paper
TLN kernels and finite-volume master trace	Published in the 2014 paper
Separate flow/action/observable discretizations	Published and scientifically required
FVN rectangular four-torus product	Derived toolkit extension
Complete theta evaluation with modular transform	Numerical implementation of the published Jacobi function
TLN restriction to Wilson flow	Current implementation restriction
TLN $N_s^3 N_t$ geometry	Derived extension plus implementation restriction to equal spatial axes
Momentum orbit reduction	Exact implementation optimization, closure checked
Spectral trace identity	Exact linear algebra reorganization
Absolute exponent merge tolerance 10^{-11}	Numerical toolkit policy
Direct requested-time exponential sum	Exact reorganization up to floating arithmetic
Fixed-order reduction without fast math	Determinism policy
Exact $c \leq 1/2$ boundary	Enforced applicability policy
Returning $C - 1$ and dividing by $1 + \Delta$	Toolkit interface convention consistent with the papers

15 Applicability limits and failure behavior

15.1 FVN

- Requires canonical four-positive-extent syntax and a nonempty positive finite one-dimensional time array.
- Implements periodic continuum gauge-field finite-volume normalization.
- Does not depend on lattice flow/action/operator discretization.
- Reproduces a rectangular-torus product extension, while the canonical displayed paper formula is hypercubic.
- Evaluates each complete theta factor through a direct or modular series to the declared relative tolerance.

15.2 TLN

- Requires periodic finite lattices with three equal spatial extents.
- Supports Wilson flow only.
- Supports Wilson or tree-level Symanzik simulation gauge action.
- Supports plaquette, tree-level Symanzik, or clover energy density.
- Is a tree-level correction; it does not remove higher-order cutoff effects.
- Rejects flow times above the exact $c = 1/2$ boundary before calculation.
- Evaluates requested times directly and rejects nonpositive or nonfinite normalization factors.

All invalid supported-code or geometry cases raise `TreeLevelCorrectionError`. At the processing boundary, that becomes a `ProcessingError`, and the ensemble is excluded with a recorded reason. An unsupported correction is therefore visible rather than silently replaced by the nearest available scheme.

Empty time arrays, malformed geometry strings, and a 1^4 TLN volume with no nonzero momentum now fail through `TreeLevelCorrectionError`. These contracts apply equally to direct and processing callers.

16 Computational characteristics

16.1 FVN cost

FVN is vectorized over requested flow times and loops over four directions plus the rapidly convergent theta terms. Its time and memory costs are $O(TK)$, where the modular transformation keeps the required term count K small. It performs no interpolation or momentum enumeration.

16.2 TLN cost

Before orbit reduction, the momentum grid has $N_s^3 N_t - 1$ nonzero points. Orbit reduction can decrease the number of representatives substantially; for 8^4, 4095 points become 174 representatives.

For each representative, the code constructs three 4×4 matrices, diagonalizes the flow matrix, solves the gauge matrix against four eigenvectors, and produces 16 spectral terms. Blocks cap intermediate memory. Exponent collapsing reduces later work when degeneracies occur.

Requested-time evaluation scales with the collapsed term count times the number of requested times. Parallelism is across times, while each result has a fixed sequential reduction. The 64-entry LRU cache makes repeated calls for the same geometry, action, observable, and collapse tolerance reuse the expensive spectrum construction.

The scientific result is exactly independent of thread count in current tests. Exponent-collapse tolerance remains a declared approximation and an appropriate convergence-sweep parameter.

17 Current validation oracles and remaining gaps

17.1 Covered through the public interface

The test suite currently verifies:

1. reports include the canonical DOI, validity domain, numerical tolerance, and implementation notes;
2. complete FVN theta evaluation at the discriminating $c = 2$ oracle;
3. TLN approaches FVN at fixed c as L/a increases;
4. an independent full-momentum, matrix-exponential TLN formula on a tiny lattice for all six gauge-action/energy-operator combinations;
5. direct TLN agreement at $t/a^2 = 10^{-4}$, where the removed spline was inaccurate;
6. exact equality across Numba thread counts;
7. exact boundary rejection plus typed failures for malformed geometry, empty times, a lattice without nonzero momentum, and non-Wilson flow.

17.2 Exact internal invariant

Every TLN spectrum computation checks Eq. (9), so orbit multiplicities must cover the complete nonzero momentum set exactly.

The full, unreduced $4^3 \times 6$ momentum sum is now an automated repository test rather than an external audit observation. It is independent of orbit reduction, spectral collapsing, and the direct-evaluation implementation.

17.3 Useful future strengthening

The following would provide more independent evidence:

- sweep FVN tolerance against a higher-precision reference across extreme rectangular aspect ratios;
- sweep TLN exponent-collapse tolerance against the independent full sum;
- test odd spatial/temporal extents and anisotropic time extent;
- extend positivity and finiteness sampling over larger supported lattices;
- compare published table or figure values when a directly matching geometry and discretization is available.

These are not reasons to discard the implementation. They distinguish an algebraically well-mapped method from a fully quantified numerical error budget.

18 Source ledger

Implemented element	Primary source	Scope of authority
Continuum FVN	Fodor et al., DOI 10.1007/JHEP11(2012)007	Hypercubic periodic-torus formula and zero/nonzero-mode terms
Finite-lattice TLN	Fodor et al., DOI 10.1007/JHEP09(2014)018	Lattice momenta, kernels, trace, finite-volume zero mode, discretization dependence
Rectangular FVN product	Direct separable extension retained from legacy implementation	Not the displayed hypercubic formula in the source
$N_s^3 N_t$ TLN volume	Direct separable extension of the finite momentum sum	The source displays an L^4 lattice
Theta modular-series evaluation	Toolkit numerical implementation	Computes the complete Jacobi function in the source
Orbit and spectral reductions	Exact combinatorics and linear algebra derived above	Optimizations of the published finite sum
Collapse tolerance, fixed-order evaluation, cache	Toolkit numerical policy	Require numerical validation, not sourced as physics

19 File map

Concern	Implementation region
Shared correction result and evidence	`tree_level/core.py`
Continuum finite-volume normalization	`tree_level/fvn.py`
Finite-lattice tree-level normalization	`tree_level/tln.py`
Aggregate correction imports	`tree_level/__init__.py`
Correction selection, division, and reporting	`stages/process.py`
Correction method adapters and legacy selection	`setup.py`
Public workflow oracles	`test_processing_interface.py`
Independent numerical correction oracles	`test_tree_level_corrections.py`

`tree_level/` paths are relative to `src/gfrgtoolkit/stages/`; `stages/process.py` and `setup.py` are relative to `src/gfrgtoolkit/`; the test file is relative to `tests/`.